

Proximity risk after go-arounds — terminal commands

Every command and option of run_proximity.py with examples. Run everything from the repository root with the virtual environment active. Method: FRAMEWORK.md.

1. Set-up (once per machine)

<pre>git clone https://github.com/sponbob3/CollisionRiskAlgo cd CollisionRiskAlgo</pre>	Get the repository.
<pre>python -m venv .venv source .venv/bin/activate pip install -r requirements.txt</pre>	Create the virtual environment and install every dependency (pandas-2.3.1 is pinned for the traffic library). # Windows: .venv\Scripts\activate
<pre>git pull</pre>	Update to the latest version later (run inside the repository).

2. Data

<pre>mkdir -p datasets/KBNA_2025 cp /path/to/daily/*.parquet datasets/KBNA_2025/</pre>	One file per day (.parquet preferred, .csv accepted) of OpenSky-style state vectors for ALL traffic around the airport, columns: timestamp, icao24, callsign, latitude, longitude, altitude, geoaltitude, vertical_rate, groundspeed, track, onground. The folder name is _; the prefix selects airports/.yaml. datasets/ is gitignored.
<pre>python run_proximity.py check-data KBNA_2025 python run_proximity.py check-data KBNA_2025 --limit 5</pre>	Data audit only: days found, columns, per-day message and aircraft counts, gaps and receiver outages, altitude references, and whether the volume meets the baseline minimum (28 days, 90 recommended). Writes output/KBNA/proximity_risk/check_data_KBNA_2025.csv.

3. New airport

<pre>python run_proximity.py new-airport KBNA</pre>	Generate airports/KBNA.yaml from the OurAirports database (runways, elevation) with the airspace block pre-filled as field elevation + 4,000 ft. Then EDIT: timezone, preset (air_carrier / training_ga), terrain, assume_arrivals_dataset, airspace.ceiling_ft_msl (verify against the sectional chart), and optionally a flows: block (which runway ends operate together).
---	---

4. Running the analysis

<pre>python run_proximity.py KBNA_2025</pre>	The complete run: go-around detection -> output/KBNA/goaround/run_NN/, then baseline (built or reused), encounters, window metrics, every event, sensitivity sweep, tables, figures, per-event plots and summary.pdf -> output/KBNA/proximity_risk/run_NN/ (same NN).
<pre>python run_proximity.py KBNA_2025 --limit 5 --no-sweep</pre>	Quick test for the first 5 days without the sweep or figures.
<pre>python run_proximity.py KBNA_2025 --workers 4</pre>	Parallel day processing (default: CPU count - 1).
<pre>python run_proximity.py KBNA_2025 --goaround-report</pre>	Also write the go-around stage's figures, narrative report and PDF, and its plateau-duration calibration histogram.

<code>python run_proximity.py KBNA_2025 --window 5</code>	Pre/post window length in minutes (default 10; the sweep runs 5 and 15 anyway).
<code>python run_proximity.py KBNA_2025 --window 15</code>	
<code>python run_proximity.py KBNA_2025 --include-ambiguous</code>	Primary event set also includes <code>ga_ambiguous</code> go-arounds.
<code>python run_proximity.py KBNA_2025 --no-event-plots</code>	Skip the one-PNG-per-go-around plots only (figures and summary.pdf are still produced).

5. Reusing runs

<code>python run_proximity.py KBNA_2025 --goaround-run</code>	Reuse output/KBNA/goaround/run_03 instead of re-running go-around detection (the proximity run still gets a new NN).
<code>ls output/KBNA/proximity_risk/cache/2025/</code>	Per-day pairwise caches (parameter-hashed). They are reused automatically by every run and sweep; delete the folder to force recomputation.

6. Baselines

<code>python run_proximity.py KBNA_2025 --baseline-only</code>	Build (or reuse) and validate the baseline, write <code>baseline_report.pdf</code> , and stop. Stored in <code>output/KBNA/proximity_risk/baseline/2025/</code> (<code>baseline.json</code> , <code>baseline_windows.parquet</code> , <code>error_model.csv</code> , <code>phase1_removed.csv</code> , <code>validation/</code>).
<code>python run_proximity.py KBNA_2025 --rebuild-baseline</code>	Force a rebuild even if the stored baseline matches the data and parameters (it is rebuilt automatically when either changes).
<code>python run_proximity.py KBNA_spring26 --baseline-score</code>	Score KBNA_2025 dataset's events against the stored KBNA_2025 baseline (same airport).

7. Checks and tests

<code>python -m pytest</code>	The whole test suite (analytic intervals vs brute force, missed passes, scenarios, pruning, probability vs Monte Carlo, baseline calibration, injected effect, go-around parity, traffic cross-check). About two minutes.
<code>python -m pytest -q -m 'not slow'</code>	Skip the end-to-end synthetic runs.
<code>python -m proximity_pipeline.validate_cpa KBNA_2025 2025-08-08 08:00:00</code>	Cross-check close approach distances against the traffic library on the named days.
<code>python tools/make_synthetic_dataset.py KBNA_synth</code>	Synthetic traffic for a smoke test without real data
<code>python run_proximity.py KBNA_synth --limit 3</code>	(--intruder-rate and --post-ga-irr inject a known effect).
<code>python tools/make_terminal_commands_pdf.py</code>	Regenerate this PDF after changing the CLI.

8. Where things land

<code>output/KBNA/goaround/run_NN/</code>	Exact go-around pipeline outputs (<code>all_approaches.csv</code> , <code>go_around_events.csv</code> , <code>summary.pdf</code> , <code>plots/</code> , <code>run_config.txt</code>).
---	--

output/KBNA/proximity_risk/run_NN/	run_config.txt, ceiling_check.csv, data_quality.csv, window_metrics.parquet, encounters.parquet/.csv, go_around_risk.csv (+ _involved), results_primary.csv, equilibrium.csv, epoch.csv, sensitivity.csv, figures/, events/, summary.pdf.
------------------------------------	---

Options summary: --goaround-run N, --rebuild-baseline, --baseline-only, --baseline-from ICAO_label, --window MIN, --include-ambiguous, --no-sensitivity, --no-plots, --no-event-plots, --goaround-report, --calibrate, --limit N, --workers N.
Sub-commands: new-airport ICAO, check-data ICAO_label [--limit N].